

MAT 336 Info Systems: Webscraping

Webscraping is a technique to extract data from a website. Today we'll extract player passing data for the 2018 regular season from nfl.com.

Start by getting familiar with this website (recommend using Google Chrome):

<https://www.nfl.com/stats/player-stats/category/passing/2018/REG/all/passingyards/DESC>

We will be interested in extracting the player's name, team url, position, completions, attempts, yards, and touchdowns.

Right-click on Ben Roethlisberger's name and select "Inspect". This opens the web developer tool, which shows the html code used for this webpage. We will pick through the html tags in order to extract the information we want. What html tag contains the information we want?

Step 1. Initialize BeautifulSoup object and download page contents

First, we need to establish the connection to the website, and create a beautiful soup object with all of the data (HTML) within the website. To do this, we'll need to import both the 'beautifulSoup' library (this is the main library we'll be using to parse the website), and the 'requests' library (this will help us establish our connection to the website).

To create a beautiful soup object (we'll name our variable 'soup'), you need to pass it the content of a website, and you can specify which type of parser this object should use to parse the website. We specify this parser as the second argument when creating a BeautifulSoup object. Our options: 'html.parser', 'lxml', 'xml', 'html5lib'. We would recommend using 'html.parser' to start with as this is normally sufficient. If you know with certainty that it's not working as expected, try a different parser.

The BeautifulSoup documentation page is incredibly helpful with gaining some basic understanding of the BeautifulSoup library:

<https://www.crummy.com/software/BeautifulSoup/bs4/doc/#contents-and-children>

```
In [1]: # import the libraries we need
import requests
from bs4 import BeautifulSoup

# have url hold the website's url
url = "https://www.nfl.com/stats/player-stats/category/passing/2018/REG/all/passing"

In [2]: # Query the website and return the html to the variable 'website'
website = requests.get(url)
```

```
website
```

```
Out[2]: <Response [200]>
```

You should see:

```
<Response [200]>
```

If instead you see

```
<Response [403]>
```

Try the following:

```
In [3]: headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
website = requests.get(url , headers=headers)
website
```

```
Out[3]: <Response [200]>
```

If that doesn't work, try it again but replacing the contents of headers with the output from typing "navigator.userAgent" in the Chrome developer console for this url.

```
In [4]: # Parse the html in the 'website' variable, and store it in a
# beautiful soup form
# NOTE: there are other parsers available (we are using html.parser)
soup = BeautifulSoup(website.content, 'html.parser')
```

```
In [ ]: # Have a look at what's inside soup. What is it?
soup
```

Step 2. Extract information from the web page's table

Now that we have a BeautifulSoup object, we can use all of the class methods available to search and extract the desired data from this object. Let's think through how we will store the information we extract from this website. We want to collect: name, teamURL, completions, attempts, yards, and touchdowns for each player that is listed under the passing stats for the 2018 regular season.

Ultimately, we will want to put this table in a **pandas dataframe**. We can make a pandas dataframe from a dictionary of lists. Alternatively, we can also make a pandas dataframe from a list of dictionaries. We will opt for the latter, that is, each element in the list is a dictionary, where each dictionary holds the information for one player. The dictionary keys will be the same for each player: 'name', 'teamURL', 'position', 'completions', 'attempts', 'yards', and 'TD'. Our data structure will look like this:

```

passing_2018stats =
[
{'name': player1 name, 'teamURL': url of team, 'position': position of
player1, 'completions': completions of player1, 'attempts': attempts of
player1, 'yards': passing yards of player1, 'TD': TDs of player1},

{'name': player2 name, 'teamURL': url of team, 'position': position
of player2, 'completions': completions of player2, 'attempts': attempts of
player2, 'yards': passing yards of player2, 'TD': TDs of player1},
etc...
]

```

When extracting information from a website, we generally retrieve the information as strings. We might ultimately want to change these data types to something else (an int, or float), but for this example, we'll just focus on getting the desired information, knowing that we could convert it to a different data type if we wanted.

From the screenshot of the NFL web page below we see the quarterback data is embedded within **td** (table datum) tags within **tr** (table row or record) tags.



Using the **'find_all'** method of our BeautifulSoup object, we can identify all of the 'tr' tags. (Note: the 'find' method will only return the first instance of the 'tr' tag.

```

In [ ]: # Let's extract the content of all the tr tags
        # (we may end up with some extraneous data)
        player_records = soup.find_all('tr')
        player_records

```

```

In [7]: # Let's look at the first record - is that Ben Roethlisberger?
        player_records[0]

```

```
Out[7]: <tr><th aria-sort="none" class="header header--no-hover">Player</th><th aria-sort="descending" class="header headerSort" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingyards/asc">Pass Yds</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingaverageyards/DESC">Yds/Att</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingattempts/DESC">Att</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingcompletions/DESC">Cmp</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingcompletionpercentage/DESC">Cmp %</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingtouchdowns/DESC">TD</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passinginterceptions/DESC">INT</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingpasserrating/DESC">Rate</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingfirstdowns/DESC">1st</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingfirstdownpercentage/DESC">1st%</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passing20plusyardseach/DESC">20+</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passing40plusyardseach/DESC">40+</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passinglong/DESC">Lng</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingsacked/DESC">Sck</a></th><th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingsackedyardslost/DESC">SckY</a></th></tr>
```

```
In [8]: # How about the 2nd record?
player_records[1]
```

```
Out[8]: <tr><td scope="row" tabindex="0"><div class="d3-o-media-object d3-o-media-object--
vertical-center"><figure class="d3-o-media-object__figure d3-o-player-headshot"><p
icture is-lazy="/t_lazy"><!--[if IE 9]><video style="display:none"><![endif]--><so
urce media="(min-width:1024px)" srcset="https://static.www.nfl.com/image/private/t
_headshot_desktop/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 1x, https://static.w
w.nfl.com/image/private/t_headshot_desktop_2x/t_lazy/f_auto/league/vziogpnyp9d0xtp
iuqhr 2x, https://static.www.nfl.com/image/private/t_headshot_desktop_3x/t_lazy/f_
auto/league/vziogpnyp9d0xtpiuqhr"/><source media="(min-width:768px)" srcset="http
s://static.www.nfl.com/image/private/t_headshot_tablet/t_lazy/f_auto/league/vziogp
nyp9d0xtpiuqhr 1x, https://static.www.nfl.com/image/private/t_headshot_tablet_2x/t
_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 2x, https://static.www.nfl.com/image/priv
ate/t_headshot_tablet_3x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr"/><source srcse
t="https://static.www.nfl.com/image/private/t_person_squared_mobile/t_lazy/f_auto/
league/vziogpnyp9d0xtpiuqhr 1x, https://static.www.nfl.com/image/private/t_person_
squared_mobile_2x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 2x, https://static.w
w.nfl.com/image/private/t_person_squared_mobile_3x/t_lazy/f_auto/league/vziogpnyp9
d0xtpiuqhr"/><!--[if IE 9]></video><![endif]--></picture></figure><div class="d3-o-media-object__
body"><a aria-label="Ben Roethlisberger profile page" class="d3-o-player-fullname
nfl-o-cta--link" href="/players/ben-roethlisberger/"> Ben Roethlisberger </a></div
></div></td><td class="selected">5129</td><td>7.6</td><td>675</td><td>452</td><td>
67</td><td>34</td><td>16</td><td>96.5</td><td>248</td><td>36.7</td><td>61</td><td>
16</td><td>97</td><td>24</td><td>166</td></tr>
```

```
In [9]: # That was him, the first record was the row of column headers.
# Let's pop that off the list of records.
header_row = player_records.pop(0)
```

```
In [10]: # Let's try to parse the header row.
# Often these bs4 results are lists. Let's loop through to see the contents
# one per line.
for item in header_row:
    print(item)
```

```

<th aria-sort="none" class="header header--no-hover">Player</th>
<th aria-sort="descending" class="header headerSort" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingyards/asc">Pass Yds</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingaverageyards/DESC">Yds/Att</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingattempts/DESC">Att</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingcompletions/DESC">Cmp</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingcompletionpercentage/DESC">Cmp %</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingtouchdowns/DESC">TD</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passinginterceptions/DESC">INT</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingpasserrating/DESC">Rate</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingfirstdowns/DESC">1st</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingfirstdownpercentage/DESC">1st%</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passing20plusyardseach/DESC">20+</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passing40plusyardseach/DESC">40+</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passinglong/DESC">Lng</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingsacked/DESC">Sck</a></th>
<th aria-sort="none" class="header" scope="col"><a href="/stats/player-stats/category/passing/2018/REG/all/passingsackedyardslost/DESC">SckY</a></th>

```

```

In [11]: # We can see the header labels in the output above.
# But let's extract them from the characters around them
# using bs4's get_text() method.
for item in header_row:
    print(item.get_text())

```

```

Player
Pass Yds
Yds/Att
Att
Cmp
Cmp %
TD
INT
Rate
1st
1st%
20+
40+
Lng
Sck
SckY

```

Let's put these headings into a list.

```
In [12]: headers = []  
for item in header_row:  
    headers.append(item.get_text())  
print(headers)
```

```
['Player', 'Pass Yds', 'Yds/Att', 'Att', 'Cmp', 'Cmp %', 'TD', 'INT', 'Rate', '1st',  
'1st%', '20+', '40+', 'Lng', 'Sck', 'SckY']
```

Step 2(a)

A good initial step is to see if we can extract the information for a single player. If we can do it for one, we should easily be able to do it for the rest. Let's start by getting Ben Roethlisberger's data.

```
In [13]: player_record = player_records[0]  
for item in player_record:  
    print(item, '\n')
```

```
<td scope="row" tabindex="0"><div class="d3-o-media-object d3-o-media-object--vertical-center"><figure class="d3-o-media-object__figure d3-o-player-headshot"><picture is-lazy="/t_lazy"><!--[if IE 9]><video style="display:none"><![endif]--><source media="(min-width:1024px)" srcset="https://static.www.nfl.com/image/private/t_headshot_desktop/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 1x, https://static.www.nfl.com/image/private/t_headshot_desktop_2x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 2x, https://static.www.nfl.com/image/private/t_headshot_desktop_3x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr"/><source media="(min-width:768px)" srcset="https://static.www.nfl.com/image/private/t_headshot_tablet/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 1x, https://static.www.nfl.com/image/private/t_headshot_tablet_2x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 2x, https://static.www.nfl.com/image/private/t_headshot_tablet_3x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr"/><source srcset="https://static.www.nfl.com/image/private/t_person_squared_mobile/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 1x, https://static.www.nfl.com/image/private/t_person_squared_mobile_2x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr 2x, https://static.www.nfl.com/image/private/t_person_squared_mobile_3x/t_lazy/f_auto/league/vziogpnyp9d0xtpiuqhr"/><!--[if IE 9]></video><![endif]--></picture></figure><div class="d3-o-media-object__body"><a aria-label="Ben Roethlisberger profile page" class="d3-o-player-fullname nfl-o-cta--link" href="/players/ben-roethlisberger/"> Ben Roethlisberger </a></div></div></td>
```

```
<td class="selected">5129</td>
```

```
<td>7.6</td>
```

```
<td>675</td>
```

```
<td>452</td>
```

```
<td>67</td>
```

```
<td>34</td>
```

```
<td>16</td>
```

```
<td>96.5</td>
```

```
<td>248</td>
```

```
<td>36.7</td>
```

```
<td>61</td>
```

```
<td>16</td>
```

```
<td>97</td>
```

```
<td>24</td>
```

```
<td>166</td>
```

```
In [14]: # As above, we can extract the data we want using get_text
# Let's put the fields in a list called player_stats.
# We use strip since the player name has blank spaces around it.
```

```

player_stats = []
for item in player_record:
    player_stats.append(item.get_text().strip())
player_stats

```

```

Out[14]: ['Ben Roethlisberger',
          '5129',
          '7.6',
          '675',
          '452',
          '67',
          '34',
          '16',
          '96.5',
          '248',
          '36.7',
          '61',
          '16',
          '97',
          '24',
          '166']

```

```

In [15]: # Now zip the headers and player stats together into a dict
player_dict = dict(zip(headers, player_stats))
player_dict

```

```

Out[15]: {'Player': 'Ben Roethlisberger',
          'Pass Yds': '5129',
          'Yds/Att': '7.6',
          'Att': '675',
          'Cmp': '452',
          'Cmp %': '67',
          'TD': '34',
          'INT': '16',
          'Rate': '96.5',
          '1st': '248',
          '1st%': '36.7',
          '20+': '61',
          '40+': '16',
          'Lng': '97',
          'Sck': '24',
          'SckY': '166'}

```

Step 2(b)

Now that we were able to find Ben Roethlisberger's data, let's loop through all rows, saving each player's information to a dictionary, and then appending that dictionary to the list 'all_player_stats'. The web data came into bs4 as text, so we are going to try to convert numeric values to float here as well.

```

In [16]: def try_float(inp):
          # Convert the numeric strings to floats now to avoid headaches later
          try:

```

```
    inp = float(inp)
except:
    pass
return inp
```

```
In [ ]: all_player_stats = []
# Loop through each row we scraped into player_records
for player_record in player_records:
    # Obtain the stats for this player
    player_stats = []
    for item in player_record:
        # get the text from the td element, strip any \n's
        stat_value = item.get_text().strip()
        # try converting the stat value to a float if possible, for future use
        stat_value = try_float(stat_value)
        # add the stat to the player_stats list
        player_stats.append(stat_value)

    # Store the stats for this player in a dictionary keyed by headers
    player_dict = dict(zip(headers, player_stats))

    # Add this player's dict to the list of player stat dicts
    all_player_stats.append(player_dict)

all_player_stats
```

Step 3. Store downloaded data in a pandas dataframe

```
In [18]: import pandas as pd
player_stats_df = pd.DataFrame(all_player_stats)
player_stats_df
```

Out[18]:

	Player	Pass Yds	Yds/Att	Att	Cmp	Cmp %	TD	INT	Rate	1st	1st%	20+
0	Ben Roethlisberger	5129.0	7.6	675.0	452.0	67.0	34.0	16.0	96.5	248.0	36.7	61.0
1	Patrick Mahomes	5097.0	8.8	580.0	383.0	66.0	50.0	12.0	113.8	237.0	40.9	75.0
2	Matt Ryan	4924.0	8.1	608.0	422.0	69.4	35.0	7.0	108.1	236.0	38.8	56.0
3	Jared Goff	4688.0	8.4	561.0	364.0	64.9	32.0	12.0	101.1	233.0	41.5	69.0
4	Andrew Luck	4593.0	7.2	639.0	430.0	67.3	39.0	15.0	98.7	236.0	36.9	53.0
5	Aaron Rodgers	4442.0	7.4	597.0	372.0	62.3	25.0	2.0	97.6	200.0	33.5	55.0
6	Tom Brady	4355.0	7.6	570.0	375.0	65.8	29.0	11.0	97.7	205.0	36.0	53.0
7	Philip Rivers	4308.0	8.5	508.0	347.0	68.3	32.0	12.0	105.5	213.0	41.9	60.0
8	Eli Manning	4299.0	7.5	576.0	380.0	66.0	21.0	11.0	92.4	206.0	35.8	57.0
9	Kirk Cousins	4298.0	7.1	606.0	425.0	70.1	30.0	10.0	99.7	218.0	36.0	47.0
10	Deshaun Watson	4165.0	8.2	505.0	345.0	68.3	26.0	9.0	103.1	202.0	40.0	51.0
11	Derek Carr	4049.0	7.3	553.0	381.0	68.9	19.0	10.0	93.9	197.0	35.6	52.0
12	Drew Brees	3992.0	8.2	489.0	364.0	74.4	32.0	5.0	115.7	199.0	40.7	58.0
13	Case Keenum	3890.0	6.6	586.0	365.0	62.3	18.0	15.0	81.2	179.0	30.6	52.0
14	Dak Prescott	3885.0	7.4	526.0	356.0	67.7	22.0	8.0	96.9	184.0	35.0	39.0
15	Matthew Stafford	3777.0	6.8	555.0	367.0	66.1	21.0	11.0	89.9	198.0	35.7	44.0
16	Baker Mayfield	3725.0	7.7	486.0	310.0	63.8	27.0	14.0	93.7	171.0	35.2	52.0
17	Russell Wilson	3448.0	8.1	427.0	280.0	65.6	35.0	7.0	110.9	156.0	36.5	47.0
18	Cam Newton	3395.0	7.2	471.0	320.0	67.9	24.0	13.0	94.2	180.0	38.2	44.0
19	Mitchell Trubisky	3223.0	7.4	434.0	289.0	66.6	24.0	12.0	95.4	151.0	34.8	40.0
20	Carson Wentz	3074.0	7.7	401.0	279.0	69.6	21.0	7.0	102.2	159.0	39.6	37.0
21	Jameis Winston	2992.0	7.9	378.0	244.0	64.6	19.0	14.0	90.2	152.0	40.2	34.0
22	Sam Darnold	2865.0	6.9	414.0	239.0	57.7	17.0	15.0	77.6	130.0	31.4	40.0
23	Blake Bortles	2718.0	6.7	403.0	243.0	60.3	13.0	11.0	79.8	128.0	31.8	36.0
24	Andy Dalton	2566.0	7.0	365.0	226.0	61.9	21.0	11.0	89.6	132.0	36.2	38.0

Step 4. Perform analysis...

Who was sacked the most of the players on this list?

```
In [19]: max_sack = player_stats_df['Sck'].max()
msplayer = player_stats_df[player_stats_df['Sck'] == max_sack]['Player']
print(f'{msplayer.iloc[0]} was sacked the most at {int(max_sack):d} times.')
```

Deshaun Watson was sacked the most at 62 times.

and more...